

LAPORAN PROYEK AKHIR
TEKNOLOGI DAN PEMROGRAMAN MOBILE



Disusun oleh :

Danendra Pandya Reswara / 123230169

Rafly Andhika Dewandaru / 123230202

Kelas: IF-C

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA
2026

HALAMAN PENGESAHAN PRESENTASI

LAPORAN PROYEK AKHIR TEKNOLOGI DAN PEMROGRAMAN MOBILE

Disusun oleh:

Nama	No.Mhs.	Tanda Tangan
Danendra Pandya Reswara	123230169	
Rafly Andhika Dewandaru	123230202	

telah dipresentasikan pada tanggal

Menyetujui
Dosen Pengampu

Bagus Muhammad Akbar, S.S.T., M.Kom

KATA PENGANTAR

Puji syukur kita panjatkan kehadiran Tuhan Yang Maha Esa yang telah memberikan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan Tugas Laporan Proyek Akhir Teknologi Dan Pemrograman Mobile ini tepat waktu. Adapun tujuan dari penulisan dari laporan ini adalah untuk memenuhi tugas pada mata kuliah Teknologi Dan Pemrograman Mobile. Selain itu, laporan ini juga bertujuan untuk menambah wawasan tentang Teknologi Dan Pemrograman Mobile, bagi para pembaca dan juga bagi penulis.

Terlebih dahulu, saya mengucapkan terima kasih kepada Bapak Bagus Muhammad Akbar S.ST., M.Kom. selaku Dosen Teknologi Dan Pemrograman Mobile yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang saya tekuni ini.

Semoga laporan ini dapat memberikan manfaat dan kontribusi yang positif bagi perkembangan ilmu pengetahuan di Teknologi Dan Pemrograman Mobile. Saya menyadari bahwa tanpa bimbingan, arahan, serta dukungan dari Pak Bagus pencapaian ini tidak akan dapat saya raih.

Akhir kata, saya mohon maaf jika terdapat kekurangan dalam laporan ini. Saya mengharapkan kritik dan saran yang membangun guna perbaiki di masa yang akan datang.

Yogyakarta, 03 Mei 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN PRESENTASI.....	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PERANCANGAN	1
1.1. Latar Belakang.....	1
1.2. Tujuan Proyek.....	1
1.3. Analisis Kebutuhan.....	2
1.4. Perancangan Sistem	3
1.5. Spesifikasi Teknis.....	6
BAB II IMPLEMENTASI	8
2.1. Deskripsi Umum Implementasi	8
2.2. Struktur Project	8
2.3. Penjelasan Source Code	11
2.3.1. Fitur Satu : Fitur Autentikasi (Login, Register, Biometrik)	12
2.3.2. Fitur Dua: Dashboard	26
2.3.3. Fitur Tiga: AI Assistant.....	30
2.3.4. Fitur Empat : Map.....	37
2.3.5. Fitur Lima : Game	52
2.3.4. Fitur Enam : Profile	60
BAB III TESTING.....	75
3.1. Metode Testing.....	75
3.2. Rencana dan Prosedur Testing	75
BAB IV KESIMPULAN.....	77
4.1. Kesimpulan	77
4.2. Saran.....	77
LAMPIRAN	79
LAMPIRAN : Source Code Github	79

BAB I

PERANCANGAN

1.1. Latar Belakang

Aplikasi mobile saat ini berkembang pesat dan tidak hanya berfungsi sebagai alat komunikasi, tetapi juga sebagai sarana informasi, hiburan, dan produktivitas dalam satu platform. Namun, banyak aplikasi yang masih berfokus pada satu fungsi saja, sehingga pengguna harus berpindah-pindah aplikasi untuk memenuhi berbagai kebutuhan mereka. Hal ini tentu kurang efisien dan menurunkan kenyamanan pengguna.

Berdasarkan permasalahan tersebut, dikembangkan sebuah aplikasi mobile berbasis Flutter yang mengintegrasikan berbagai fitur dalam satu platform, seperti informasi berita, peta lokasi, konversi, game, serta asisten berbasis AI. Selain itu, aplikasi ini juga memanfaatkan sensor pada perangkat seperti sensor cahaya dan proximity untuk meningkatkan interaksi dan pengalaman pengguna.

Dengan adanya aplikasi ini, diharapkan pengguna dapat memperoleh berbagai layanan secara praktis dalam satu aplikasi, sehingga meningkatkan efisiensi, kenyamanan, dan pengalaman penggunaan teknologi mobile secara keseluruhan.

1.2. Tujuan Proyek

Adapun tujuan dari pengembangan aplikasi ini adalah sebagai berikut:

- Mengembangkan aplikasi mobile berbasis Flutter yang multifungsi dalam satu platform
- Mengintegrasikan berbagai fitur seperti berita, peta, konversi, game, dan AI assistant
- Meningkatkan efisiensi pengguna dengan mengurangi kebutuhan berpindah aplikasi
- Memanfaatkan sensor perangkat (cahaya dan proximity) untuk interaksi yang lebih responsif
- Memberikan pengalaman pengguna (user experience) yang lebih menarik dan interaktif
- Menyediakan akses informasi dan hiburan secara praktis dalam satu aplikasi
- Menerapkan teknologi modern (API, AI, dan sensor) dalam pengembangan aplikasi mobile

1.3. Analisis Kebutuhan

Analisis kebutuhan aplikasi ini dibagi menjadi dua bagian utama, yaitu kebutuhan fungsional dan kebutuhan non-fungsional.

1.3.1. Kebutuhan Fungsional:

- Sistem dapat menampilkan halaman dashboard dengan navigasi beberapa menu
- Sistem dapat menampilkan berita (news) dari API dan melakukan refresh data
- Sistem menyediakan fitur pencarian, filter, dan bookmark pada menu encyclopedia
- Sistem menyediakan fitur AI Assistant untuk menjawab pertanyaan pengguna
- Sistem dapat melakukan konversi (mata uang dan waktu)
- Sistem dapat menampilkan peta dan lokasi (misalnya lapangan futsal terdekat)
- Sistem dapat menampilkan rute dari lokasi pengguna ke tujuan
- Sistem menyediakan mini game dengan skor dan sistem nyawa
- Sistem dapat mendeteksi sensor cahaya dan proximity untuk interaksi tambahan
- Sistem menyediakan halaman profil pengguna
- Sistem dapat menyimpan data tertentu (seperti bookmark atau preferensi pengguna)

1.3.2. Kebutuhan Non-Fungsional:

- Kinerja (Performance): Aplikasi harus berjalan lancar tanpa lag pada perangkat mobile
- Keamanan (Security): Data pengguna dan akses API harus terlindungi dengan baik
- Usability: Tampilan aplikasi harus mudah digunakan dan user-friendly
- Reliabilitas: Aplikasi harus stabil dan tidak sering crash
- Kompatibilitas: Aplikasi dapat berjalan di berbagai perangkat Android
- Responsiveness: Waktu respon aplikasi cepat saat membuka fitur atau mengambil data
- Maintainability: Kode program mudah dikembangkan dan dipelihara (menggunakan struktur seperti GetX/MVC)
- Scalability: Sistem dapat dikembangkan dengan penambahan fitur baru di masa depan
- Efisiensi Resource: Penggunaan baterai dan memori harus optimal
- Aksesibilitas: Aplikasi dapat digunakan oleh berbagai kalangan pengguna dengan mudah

1.4. Perancangan Sistem

• **Arsitektur Aplikasi**

Aplikasi FootyHub dirancang menggunakan pola arsitektur MVC (Model-View-Controller) yang dikombinasikan dengan pendekatanGetX pada Flutter. Arsitektur ini digunakan untuk memisahkan antara data, tampilan, dan logika aplikasi agar lebih terstruktur, mudah dikembangkan, serta memudahkan proses pemeliharaan sistem.

Dengan menggunakan arsitektur ini, setiap komponen memiliki tanggung jawab masing-masing sehingga mengurangi ketergantungan antar bagian (low coupling) dan meningkatkan keterbacaan kode.

a) Model

Berisi struktur data dan pengelolaan data yang digunakan dalam aplikasi seperti:

- AuthApiService (komunikasi login & register ke backend)
- StorageUtil (penyimpanan token JWT dan data user)
- HiveProvider (penyimpanan data lokal seperti status biometrik)
- SharedPreferences (menyimpan data sederhana seperti username terakhir login)
- Model data API (berita, user, dll dalam bentuk JSON)

Semua pengelolaan data diletakkan di folder seperti:

lib/data/services, lib/data/locals, dan lib/core/utils

b) View

Terdiri dari halaman-halaman Flutter seperti:

- Halaman Login dan Register (login_view.dart, register_view.dart)
- Halaman Dashboard (dashboard_view.dart)
- Halaman Home & News (home_view.dart, news_view.dart)
- Halaman Encyclopedia (encyclopedia_view.dart)
- Halaman AI Assistant (ai_view.dart)
- Halaman Konversi (conversion_view.dart)
- Halaman Maps (maps_view.dart)
- Halaman Game (game_view.dart)
- Halaman Profil (profile_view.dart)

Semua view diletakkan di folder:

lib/modules/.../views

c) Controller

Mengatur logika aplikasi seperti:

- `auth_controller.dart` untuk autentikasi (login, register, biometrik)
- `dashboard_controller.dart` untuk navigasi dan sensor
- `home_controller.dart` untuk data beranda
- `news_controller.dart` untuk pengambilan berita dari API
- `encyclopedia_controller.dart` untuk search, filter, dan bookmark
- `ai_controller.dart` untuk komunikasi dengan AI
- `conversion_controller.dart` untuk perhitungan konversi
- `maps_controller.dart` untuk lokasi dan peta
- `game_controller.dart` untuk logika permainan
- `profile_controller.dart` untuk data pengguna

Semua controller diletakkan di folder:

`lib/modules/.../controllers`

- **flowchart**

Aplikasi dimulai dari proses inisialisasi saat pertama kali dijalankan. Pada tahap ini, sistem akan melakukan pengecekan sesi lokal pengguna untuk mengetahui apakah pengguna sudah pernah login sebelumnya atau belum. Jika sesi masih aktif, maka pengguna akan langsung diarahkan ke halaman dashboard utama. Sebaliknya, jika tidak ditemukan sesi aktif, pengguna akan diarahkan ke halaman login.

Pada halaman login, pengguna diberikan dua metode autentikasi, yaitu menggunakan email dan password atau menggunakan biometrik seperti sidik jari atau pengenalan wajah. Jika pengguna memilih metode email dan password, maka sistem akan memproses input kredensial tersebut dan melakukan validasi melalui API. Jika pengguna memilih metode biometrik, maka sistem akan melakukan pemindaian terlebih dahulu sebelum masuk ke tahap validasi. Setelah proses validasi berhasil, sistem akan melakukan enkripsi dan penyimpanan sesi secara lokal, kemudian mengarahkan pengguna ke dashboard utama.

Pada halaman dashboard, terdapat bilah navigasi bawah yang berfungsi sebagai pusat akses ke berbagai fitur utama aplikasi. Menu pertama adalah beranda, yang berfungsi untuk menampilkan informasi terkait sepak bola seperti berita dan jadwal pertandingan. Data pada menu ini diambil melalui API sepak bola dan juga memanfaatkan sensor cahaya perangkat untuk menyesuaikan tampilan.

Menu kedua adalah ensiklopedia, yang menyediakan informasi terkait klub dan liga sepak bola. Sistem akan mengambil data dari API, kemudian menampilkan daftar tim yang dapat dipilih pengguna untuk melihat detail informasi lebih lanjut.

Menu ketiga adalah tools atau konversi, yang menyediakan fitur tambahan seperti konversi mata uang menggunakan API FloatRates, tampilan jam dunia, serta pemanfaatan sensor accelerometer untuk melakukan refresh data dengan gerakan perangkat.

Menu keempat adalah Pundit AI, yaitu fitur berbasis kecerdasan buatan yang memungkinkan pengguna untuk berinteraksi melalui chat. Sistem akan mengirimkan prompt ke API Cohere dan menampilkan respon yang dihasilkan secara interaktif.

Menu kelima adalah maps, yang menyediakan fitur peta berbasis lokasi. Sistem akan meminta izin akses GPS pengguna, kemudian menampilkan peta yang berisi lokasi lapangan futsal atau stadion. Selain itu, terdapat fitur pencarian untuk menemukan lokasi tertentu.

Menu keenam adalah profil, yang menampilkan identitas pengguna seperti nama dan NIM. Pada menu ini juga tersedia formulir evaluasi serta fitur logout.

Proses logout dilakukan ketika pengguna memilih aksi keluar. Sistem akan menghapus data sesi yang tersimpan secara lokal, kemudian mengarahkan pengguna kembali ke halaman login.

Package Manager	pubspec.yaml (Flutter default)
Paket Eksternal	get, http, shared_preferences, hive, local_auth, flutter_map, latlong2, intl, google_fonts
Versi SDK	Flutter 3.x.x (stable)

Dengan spesifikasi teknis ini, aplikasi diharapkan dapat berjalan lancar dan optimal pada berbagai perangkat mobile modern yang banyak digunakan saat ini.

BAB II

IMPLEMENTASI

2.1. Deskripsi Umum Implementasi

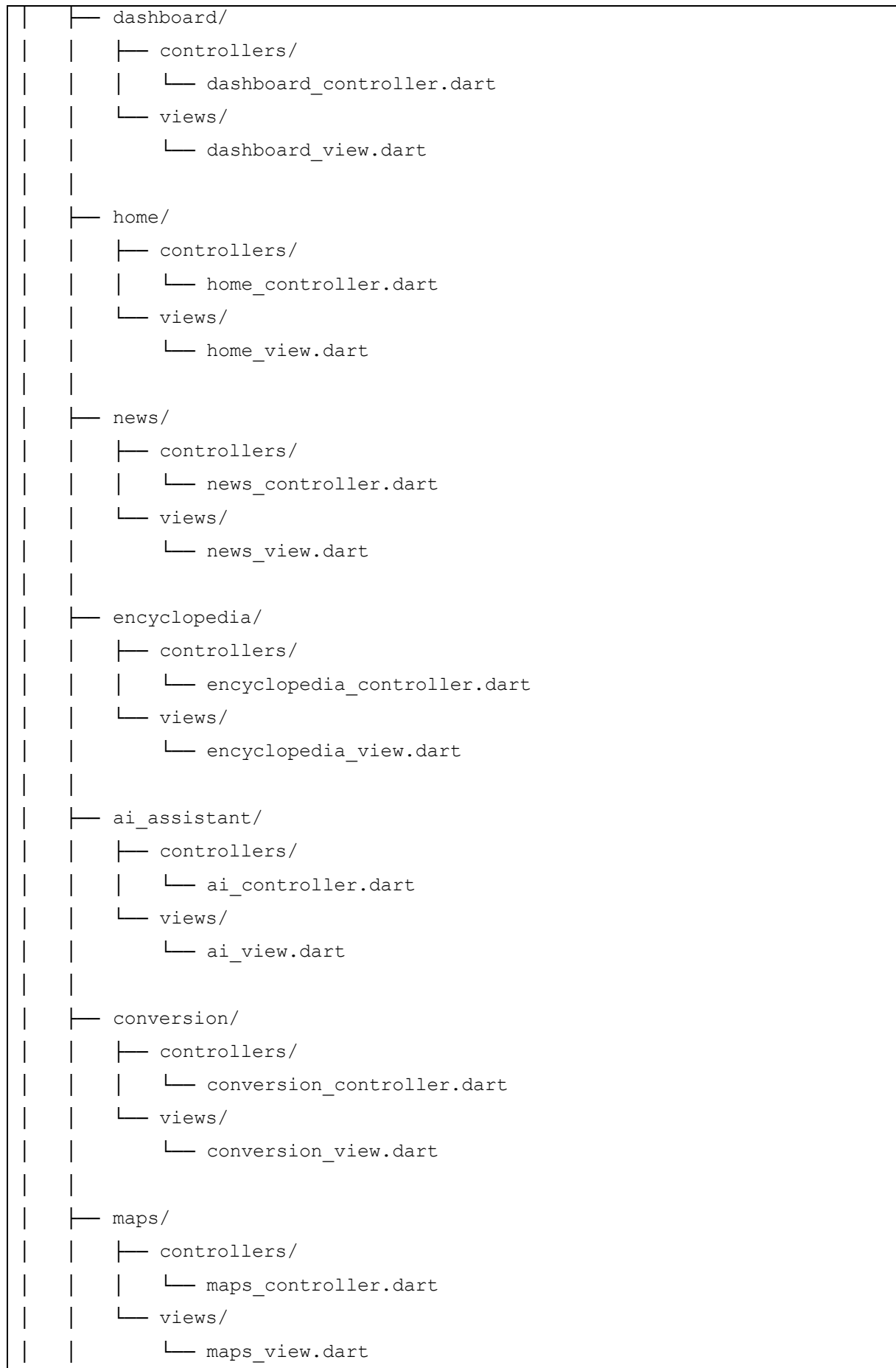
Proyek ini diimplementasikan sebagai aplikasi mobile menggunakan framework Flutter dengan bahasa pemrograman Dart. Arsitektur yang digunakan adalah pendekatan modular dengan pemisahan antara tampilan (view) dan logika (controller), sehingga memudahkan pengembangan dan pemeliharaan aplikasi. Pengelolaan state dan navigasi memanfaatkan framework seperti GetX agar aplikasi lebih terstruktur dan efisien.

Aplikasi ini mengintegrasikan berbagai fitur utama, seperti pengambilan data berita melalui API, penggunaan layanan peta untuk menampilkan lokasi dan rute, serta integrasi AI Assistant untuk memberikan respon otomatis terhadap pertanyaan pengguna. Selain itu, aplikasi juga memanfaatkan sensor perangkat seperti sensor cahaya dan proximity untuk meningkatkan interaksi pengguna.

Setiap fitur dikembangkan dalam modul terpisah, seperti modul berita, konversi, peta, game, dan profil pengguna. Data yang diperlukan diambil dari API eksternal maupun disimpan secara lokal untuk kebutuhan tertentu seperti bookmark. Dengan pendekatan ini, aplikasi menjadi lebih fleksibel, mudah dikembangkan, serta mampu memberikan pengalaman pengguna yang interaktif dan responsif.

2.2. Struktur Project

```
lib/  
|  
├─ main.dart  
|  
├─ core/  
|   ├─ constants/  
|   |   └─ app_colors.dart  
|   ├─ services/  
|   |   ├─ api_service.dart  
|   |   └─ sensor_service.dart  
|   └─ utils/  
|       └─ helpers.dart  
|  
├─ modules/  
|   |
```





Gambar 2.1. Struktur Project

Struktur project pada aplikasi ini disusun menggunakan pendekatan arsitektur modular dengan pemisahan yang jelas antara komponen utama seperti tampilan (view), logika (controller), dan layanan pendukung (services). Pendekatan ini bertujuan untuk meningkatkan keteraturan kode, memudahkan proses pengembangan, serta mempermudah pemeliharaan dan pengembangan fitur di masa mendatang.

Pada bagian utama terdapat file `main.dart` yang berfungsi sebagai titik awal (entry point) dari aplikasi. File ini bertanggung jawab untuk menjalankan aplikasi serta menginisialisasi konfigurasi awal seperti routing dan dependency yang dibutuhkan.

Folder `core/` berisi komponen dasar yang digunakan secara global di seluruh aplikasi. Di dalamnya terdapat beberapa bagian penting, seperti `constants/` yang menyimpan nilai tetap seperti warna (`app_colors.dart`), sehingga tampilan aplikasi menjadi konsisten. Kemudian terdapat `services/` yang berisi logika untuk berkomunikasi dengan API eksternal serta mengelola sensor perangkat seperti sensor cahaya dan proximity. Selain itu, terdapat `utils/` yang berisi fungsi-fungsi bantuan (helper) untuk mempermudah proses pengolahan data atau kebutuhan umum lainnya.

Folder `modules/` merupakan bagian utama dari aplikasi yang berisi berbagai fitur yang tersedia. Setiap fitur dipisahkan ke dalam modul tersendiri, seperti `dashboard`, `home`, `news`, `encyclopedia`, `AI assistant`, `conversion`, `maps`, `game`, dan `profile`. Setiap modul memiliki

struktur yang sama, yaitu terdiri dari folder controllers/ dan views/. Folder controllers berisi logika bisnis dan pengelolaan data menggunakan GetX, sedangkan folder views berisi tampilan antarmuka yang ditampilkan kepada pengguna. Pemisahan ini membuat kode lebih terorganisir dan memudahkan pengembangan secara terpisah tanpa mengganggu bagian lain.

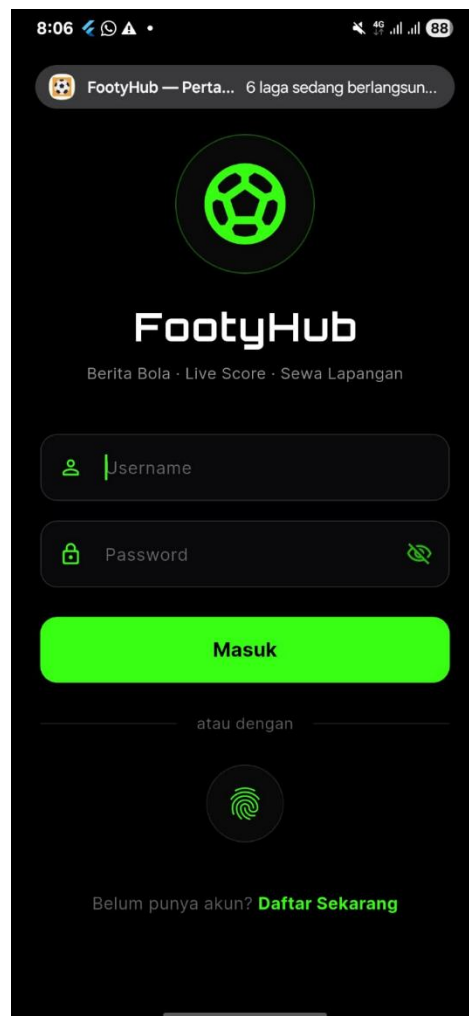
Selanjutnya, folder routes/ digunakan untuk mengatur navigasi antar halaman dalam aplikasi. File seperti app_pages.dart dan app_routes.dart berfungsi untuk mendefinisikan daftar halaman serta jalur navigasi, sehingga perpindahan antar fitur dapat dilakukan dengan mudah dan terstruktur.

Dengan struktur seperti ini, aplikasi menjadi lebih mudah dikembangkan karena setiap fitur berdiri sendiri (modular), sehingga jika terjadi perubahan atau penambahan fitur baru, pengembang tidak perlu mengubah keseluruhan sistem. Selain itu, penggunaan pendekatan ini juga mendukung skalabilitas aplikasi, menjaga keterbacaan kode, serta meningkatkan efisiensi kerja tim dalam pengembangan aplikasi.

2.3. Penjelasan Source Code

Pada aplikasi ini, struktur kode mengikuti pola pemisahan tanggung jawab dengan menggunakan Controller dan Bindings yang dikendalikan oleh GetX untuk manajemen state dan dependensi. Setiap fitur utama aplikasi diimplementasikan dalam beberapa bagian kode yang saling terhubung.

2.3.1. Fitur Satu : Fitur Autentikasi (Login, Register, Biometrik)



Gambar 2.2. Halaman Login dan Register

Auth Controller

Mengatur seluruh logika autentikasi seperti:

- Registrasi user ke server melalui API
- Login dan menerima token JWT
- Menyimpan sesi login ke local storage
- Login menggunakan biometrik (fingerprint)
- Mengatur state seperti loading dan error


```

import 'package:get/get.dart';
import 'package:local_auth/local_auth.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../core/utils/storage_util.dart';
import '../data/locals/hive_provider.dart';
import '../data/services/auth_api_service.dart';

class AuthController extends GetxController {
  var isLoading = false.obs;
  var errorMessage = ''.obs;

  var isPasswordHidden = true.obs;
  var isConfirmPasswordHidden = true.obs;

  final LocalAuthentication auth = LocalAuthentication();

  void togglePasswordVisibility() {
    isPasswordHidden.value = !isPasswordHidden.value;
  }

  void toggleConfirmPasswordVisibility() {
    isConfirmPasswordHidden.value = !isConfirmPasswordHidden.value;
  }

  Future<bool> register(
    String name,
    String nim,
    String username,
    String password,
  ) async {
    if (name.isEmpty || nim.isEmpty || username.isEmpty ||
password.isEmpty) {
      errorMessage.value = 'Semua kolom harus diisi';
      return false;
    }

    isLoading.value = true;
    try {
      final res = await AuthApiService.register(
        username: username.trim(),

```

```

        password: password,
        name: name.trim(),
        nim: nim.trim(),
    );
    if (res != null && res['token'] != null) {
        final token = res['token'] as String;
        final user = res['user'] as Map<String, dynamic>? ?? {};
        await StorageUtil.saveJwtSession(
            token: token,
            username: user['username']?.toString() ?? username.trim(),
            name: user['name']?.toString() ?? name,
            nim: user['nim']?.toString() ?? nim,
        );
        errorMessage.value = '';
        return true;
    }
    errorMessage.value =
        res?['error']?.toString() ?? 'Registrasi gagal - cek server &
URL auth.';
    return false;
} catch (e) {
    errorMessage.value = 'Gagal mendaftar: $e';
    return false;
} finally {
    isLoading.value = false;
}
}

Future<bool> login(String username, String password) async {
    if (username.isEmpty || password.isEmpty) {
        errorMessage.value = 'Username dan password harus diisi';
        return false;
    }

    isLoading.value = true;
    try {
        final res = await AuthApiService.login(
            username: username.trim(),
            password: password,
        );
        if (res != null && res['token'] != null) {

```

```

        final token = res['token'] as String;
        final user = res['user'] as Map<String, dynamic>? ?? {};
        await StorageUtil.saveJwtSession(
            token: token,
            username: user['username']?.toString() ?? username.trim(),
            name: user['name']?.toString() ?? 'User',
            nim: user['nim']?.toString() ?? '-',
        );
        errorMessage.value = '';
        return true;
    }
    errorMessage.value =
        res?['error']?.toString() ?? 'Login gagal – cek server /
kredensial.';
    return false;
} catch (e) {
    errorMessage.value = 'Terjadi kesalahan: $e';
    return false;
} finally {
    isLoading.value = false;
}
}

Future<bool> loginWithBiometric() async {
    try {
        final prefs = await SharedPreferences.getInstance();
        final lastUser = prefs.getString(StorageUtil.keyLastLoginUsername);

        if (lastUser == null || lastUser.isEmpty) {
            errorMessage.value =
                'Login manual dulu, lalu aktifkan biometrik di menu Akun.';
            return false;
        }

        if (!HiveProvider.getBiometricStatus(lastUser)) {
            errorMessage.value =
                'Biometrik belum diaktifkan. Aktifkan di menu Akun setelah
login.';
            return false;
        }
    }
}

```

```

    final sessionOk = await StorageUtil.isValidSession();
    if (!sessionOk) {
      errorMessage.value =
        'Sesi habis. Login dengan password terlebih dahulu.';
      return false;
    }

    bool canCheckBiometrics = await auth.canCheckBiometrics;
    bool isDeviceSupported = await auth.isDeviceSupported();

    if (!canCheckBiometrics || !isDeviceSupported) {
      errorMessage.value = 'Perangkat tidak mendukung biometrik.';
      return false;
    }

    bool didAuthenticate = await auth.authenticate(
      localizedReason: 'Autentikasi untuk masuk ke FootyHub',
      biometricOnly: true,
    );

    return didAuthenticate;
  } catch (e) {
    errorMessage.value = 'Biometrik dibatalkan atau gagal.';
    return false;
  }
}

```

Login View

Menyediakan form login (username & password), tombol login, serta opsi login biometrik. Jika login berhasil, user diarahkan ke dashboard.

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';
import '../routes/app_routes.dart';
import 'widgets/custom_textfield.dart';
import 'auth_controller.dart';

```

```

class LoginView extends StatelessWidget {
  const LoginView({super.key});

  @override
  Widget build(BuildContext context) {
    final AuthController authC = Get.find<AuthController>();

    final TextEditingController usernameController =
TextEditingController();
    final TextEditingController passwordController =
TextEditingController();

    return Scaffold(
      backgroundColor: Colors.white,
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical:
40),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.center,
            children: [
              const Icon(Icons.sports_soccer, size: 100, color:
AppColors.primary),
              Text(
                'FootyHub',
                style: GoogleFonts.inter(
                  fontSize: 28,
                  fontWeight: FontWeight.bold,
                  color: AppColors.primary,
                ),
              ),
              const SizedBox(height: 6),
              Text(
                'Berita bola · live score · lapangan',
                style: GoogleFonts.inter(
                  fontSize: 14,
                  color: AppColors.secondary,
                ),
              ),
              const SizedBox(height: 32),
            ],
          ),
        ),
      ),
    );
  }
}

```

```

CustomTextField(
  controller: usernameController,
  hintText: 'Username',
  prefixIcon: Icons.person_outline,
),
Obx(
  () => CustomTextField(
    controller: passwordController,
    hintText: 'Password',
    prefixIcon: Icons.lock_outline,
    isPassword: true,
    obscureText: authC.isPasswordHidden.value,
    onTogglePassword: () =>
authC.togglePasswordVisibility(),
  ),
),
const SizedBox(height: 8),
Obx(
  () => SizedBox(
    width: double.infinity,
    height: 54,
    child: ElevatedButton(
      onPressed: authC.isLoading.value
        ? null
        : () async {
            bool success = await authC.login(
              usernameController.text,
              passwordController.text,
            );
            if (success) {
              Get.offAllNamed(Routes.DASHBOARD);
            } else {
              Get.snackbar(
                'Login gagal',
                authC.errorMessage.value,
                backgroundColor: Colors.redAccent,
                colorText: Colors.white,
              );
            }
          },
      style: ElevatedButton.styleFrom(

```

```

        backgroundColor: AppColors.primary,
        shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(14),
        ),
        elevation: 0,
    ),
    child: authC.isLoading.value
        ? const CircularProgressIndicator(color:
Colors.white)
        : Text(
            'Login',
            style: GoogleFonts.inter(
                fontSize: 16,
                fontWeight: FontWeight.w600,
                color: Colors.white,
            ),
        ),
    ),
),
const SizedBox(height: 24),
Row(
    children: [
        const Expanded(
            child: Divider(color: AppColors.tfBorder, thickness:
1),
        ),
        Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Text(
                'atau',
                style: GoogleFonts.inter(color:
AppColors.tfPlaceholder),
            ),
        ),
        const Expanded(
            child: Divider(color: AppColors.tfBorder, thickness:
1),
        ),
    ],
),

```

```

const SizedBox(height: 24),
Container(
  height: 64,
  width: 64,
  decoration: const BoxDecoration(
    color: AppColors.tfBackground,
    shape: BoxShape.circle,
  ),
  child: IconButton(
    icon: const Icon(
      Icons.fingerprint,
      color: AppColors.primary,
      size: 32,
    ),
    onPressed: () async {
      bool success = await authC.loginWithBiometric();
      if (success) {
        Get.offAllNamed(Routes.DASHBOARD);
      } else if (authC.errorMessage.value.isNotEmpty) {
        Get.snackbar(
          'Biometrik',
          authC.errorMessage.value,
          backgroundColor: Colors.redAccent,
          colorText: Colors.white,
          snackPosition: SnackPosition.BOTTOM,
          margin: const EdgeInsets.all(24),
        );
      }
    },
  ),
),
const SizedBox(height: 30),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(
      'Belum punya akun? ',
      style: GoogleFonts.inter(color:
AppColors.tfPlaceholder),
    ),
    GestureDetector(

```



```

        onTap: () => Get.toNamed(Routes.REGISTER),
        child: Text(
          'Daftar',
          style: GoogleFonts.inter(
            color: AppColors.primary,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  ],
),
),
);
}
})

```

Register View

Menyediakan form pendaftaran (nama, NIM, username, password). Setelah berhasil, user langsung masuk ke aplikasi.

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';
import '../routes/app_routes.dart';
import 'widgets/custom_textfield.dart';
import 'auth_controller.dart';

class RegisterView extends StatelessWidget {
  const RegisterView({super.key});

  @override
  Widget build(BuildContext context) {
    final AuthController authC = Get.find<AuthController>();

    final TextEditingController nameController = TextEditingController();
    final TextEditingController nimController = TextEditingController();
  }
}

```

```

        final TextEditingController usernameController =
TextEditingController();

        final TextEditingController passwordController =
TextEditingController();

        final TextEditingController confirmPasswordController =
            TextEditingController();

return Scaffold(
    backgroundColor: Colors.white,
    body: SafeArea(
        child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical:
40),
            child: Column(
                children: [
                    const Icon(Icons.sports_soccer, size: 88, color:
AppColors.primary),
                    Text(
                        'FootyHub',
                        style: GoogleFonts.inter(
                            fontSize: 26,
                            fontWeight: FontWeight.bold,
                            color: AppColors.primary,
                        ),
                    ),
                    Text(
                        'Daftar akun (backend bcrypt + JWT)',
                        style: GoogleFonts.inter(
                            fontSize: 13,
                            color: AppColors.tfPlaceholder,
                        ),
                    ),
                    const SizedBox(height: 28),
                    CustomTextField(
                        controller: nameController,
                        hintText: 'Nama lengkap',
                        prefixIcon: Icons.person_outline,
                    ),
                    CustomTextField(
                        controller: nimController,
                        hintText: 'NIM',

```

```

        prefixIcon: Icons.badge_outlined,
      ),
      CustomTextField(
        controller: usernameController,
        hintText: 'Username',
        prefixIcon: Icons.alternate_email,
      ),
      Obx(
        () => CustomTextField(
          controller: passwordController,
          hintText: 'Password',
          prefixIcon: Icons.lock_outline,
          isPassword: true,
          obscureText: authC.isPasswordHidden.value,
          onTogglePassword: () =>
authC.togglePasswordVisibility(),
        ),
      ),
      Obx(
        () => CustomTextField(
          controller: confirmPasswordController,
          hintText: 'Konfirmasi password',
          prefixIcon: Icons.lock_outline,
          isPassword: true,
          obscureText: authC.isConfirmPasswordHidden.value,
          onTogglePassword: () =>
            authC.toggleConfirmPasswordVisibility(),
        ),
      ),
      const SizedBox(height: 18),
      Obx(
        () => SizedBox(
          width: double.infinity,
          height: 54,
          child: ElevatedButton(
            onPressed: authC.isLoading.value
              ? null
              : () async {
                  if (passwordController.text !=
confirmPasswordController.text) {
                    Get.snackbar(

```

```

        'Error',
        'Konfirmasi password tidak cocok',
    );
    return;
}
bool success = await authC.register(
    nameController.text,
    nimController.text,
    usernameController.text,
    passwordController.text,
);
if (success) {
    Get.offAllNamed(Routes.DASHBOARD);
    Get.snackbar(
        'Berhasil',
        'Akun dibuat dan sesi JWT aktif.',
    );
} else {
    Get.snackbar('Gagal',
authC.errorMessage.value);
    }
},
style: ElevatedButton.styleFrom(
    backgroundColor: AppColors.primary,
    shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
    ),
),
child: authC.isLoading.value
    ? const CircularProgressIndicator(color:
Colors.white)
    : Text(
        'Daftar',
        style: GoogleFonts.inter(
            fontSize: 16,
            fontWeight: FontWeight.w600,
            color: Colors.white,
        ),
    ),
),
),
),

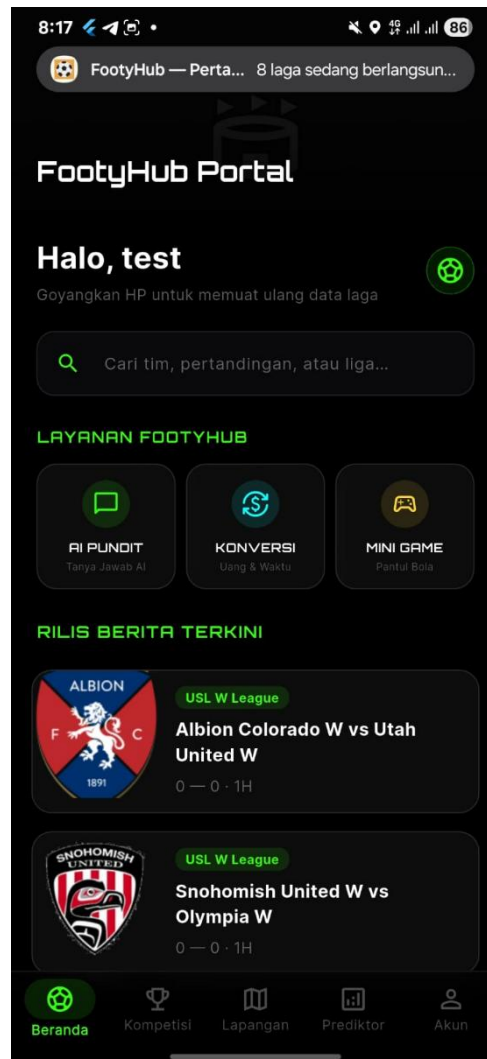
```

```

        ),
        const SizedBox(height: 18),
        Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
                Text(
                    'Sudah punya akun? ',
                    style: GoogleFonts.inter(color:
AppColors.tfPlaceholder),
                ),
                GestureDetector(
                    onTap: () => Get.back(),
                    child: Text(
                        'Login',
                        style: GoogleFonts.inter(
                            color: AppColors.primary,
                            fontWeight: FontWeight.w600,
                        ),
                    ),
                ),
            ],
        ),
    ],
),
),
),
);
}
}

```

2.3.2. Fitur Dua: Dashboard



Gambar 2.3. Halaman Dashboard

Dashboard berfungsi sebagai pusat navigasi yang menghubungkan seluruh fitur dalam aplikasi. Pada bagian controller, terdapat logika untuk mengatur menu atau tab yang sedang aktif. Dashboard juga terintegrasi dengan sensor perangkat seperti sensor cahaya dan proximity untuk memberikan interaksi tambahan.

Pada bagian view, dashboard menampilkan tampilan navigasi seperti bottom navigation bar yang memungkinkan pengguna berpindah antar fitur dengan mudah. Dengan adanya dashboard, pengguna dapat mengakses seluruh fitur dalam satu halaman utama tanpa kesulitan.

Dashboard Controller

```
import 'dart:async';
import 'package:get/get.dart';
import 'package:light/light.dart';
import 'package:proximity_sensor/proximity_sensor.dart';
import 'package:flutter/foundation.dart' as foundation;

class DashboardController extends GetxController {
  var tabIndex = 0.obs;

  // Sensors
  Light? _light;
  StreamSubscription? _lightSubscription;
  bool _hasWarnedLight = false;

  StreamSubscription? _proximitySubscription;
  var isProximityNear = false.obs;

  @override
  void onInit() {
    super.onInit();
    _initLightSensor();
    _initProximitySensor();
  }

  void _initLightSensor() {
    try {
      _light = Light();
      _lightSubscription = _light?.lightSensorStream.listen((luxValue) {
        if (luxValue < 5 && !_hasWarnedLight) {
          _hasWarnedLight = true;
          Get.snackbar(
            'Peringatan Sensor Cahaya',
            'Kondisi sekitar sangat gelap ($luxValue lux). Jaga jarak mata Anda dari layar!',
            snackPosition: SnackPosition.TOP,
            duration: const Duration(seconds: 4),
          );
        } else if (luxValue > 20) {
          _hasWarnedLight = false;
        }
      });
    } catch (e) {
      // Handle error
    }
  }

  void _initProximitySensor() {
    try {
      _proximitySubscription = ProximitySensor().proximityStream.listen((distance) {
        if (distance < 100) {
          isProximityNear.value = true;
        } else {
          isProximityNear.value = false;
        }
      });
    } catch (e) {
      // Handle error
    }
  }
}
```

```

        }
    });
} catch (_) {}
}

void _initProximitySensor() {
    try {
        _proximitySubscription = ProximitySensor.events.listen((int event)
{
        // value > 0 means near
        isProximityNear.value = (event > 0);
    });
} catch (_) {}
}

void changeTabIndex(int index) {
    tabIndex.value = index;
}

@override
void onClose() {
    _lightSubscription?.cancel();
    _proximitySubscription?.cancel();
    super.onClose();
}
}

```

Dashboard View

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'dashboard_controller.dart';
import '../home/home_view.dart';
import '../home/conversion_view.dart';
import '../maps/maps_view.dart';
import '../pundit/pundit_view.dart';
import '../profile/profile_view.dart';

class DashboardView extends StatelessWidget {
    const DashboardView({super.key});

```



```

static const _labels = ['Beranda', 'Konversi', 'Peta', 'Pundit', 'Akun'];

@override
Widget build(BuildContext context) {
  final DashboardController dash = Get.find<DashboardController>();

  return Scaffold(
    body: Stack(
      children: [
        Obx(
          () => IndexedStack(
            index: dash.tabIndex.value,
            children: const [
              HomeView(),
              ConversionView(isFromDashboard: true),
              MapsView(showBack: false),
              PunditView(),
              ProfileView(),
            ],
          ),
        Obx(
          () => dash.isProximityNear.value
            ? Container(
                color: Colors.black,
                width: double.infinity,
                height: double.infinity,
                child: const Center(
                  child: Icon(Icons.screen_lock_portrait, color:
Colors.white24, size: 100),
                ),
              )
            : const SizedBox.shrink(),
        ),
      ],
    ),
    bottomNavigationBar: Obx(
      () => NavigationBar(
        selectedIndex: dash.tabIndex.value,
        onDestinationSelected: dash.changeTabIndex,

```

```

        destinations: [
            for (var i = 0; i < _labels.length; i++)
                NavigationDestination(
                    icon: Icon(_iconFor(i, false)),
                    selectedIcon: Icon(_iconFor(i, true)),
                    label: _labels[i],
                ),
        ],
    ),
),
);
}

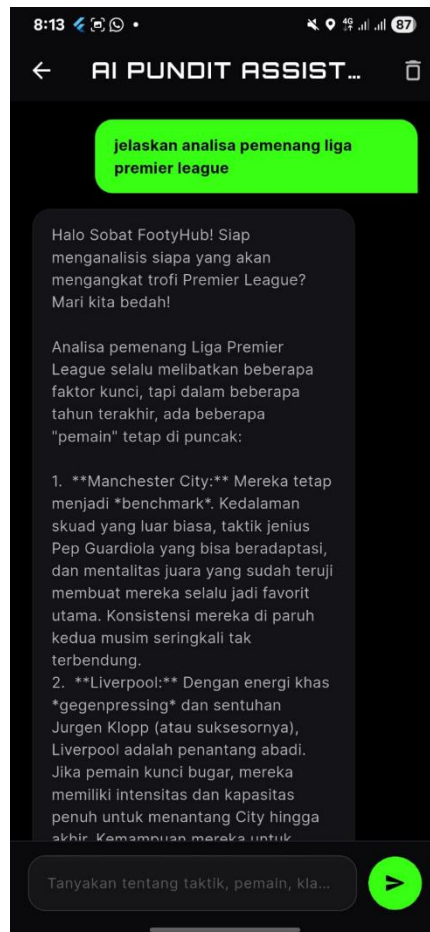
IconData _iconFor(int i, bool sel) {
    switch (i) {
        case 0:
            return sel ? Icons.home : Icons.home_outlined;
        case 1:
            return sel ? Icons.calculate : Icons.calculate_outlined;
        case 2:
            return sel ? Icons.map : Icons.map_outlined;
        case 3:
            return sel ? Icons.chat : Icons.chat_outlined;
        default:
            return sel ? Icons.person : Icons.person_outline;
    }
}
}
}

```

2.3.3. Fitur Tiga: AI Assistant

Fitur AI Assistant memungkinkan pengguna untuk berinteraksi dengan sistem menggunakan pertanyaan. Controller akan mengirimkan input pengguna ke API AI, kemudian menerima respon yang akan ditampilkan kembali kepada pengguna.

View dirancang menyerupai tampilan chat, sehingga interaksi terasa natural seperti menggunakan aplikasi pesan. Fitur ini memberikan nilai tambah karena pengguna dapat memperoleh informasi atau bantuan secara langsung melalui AI.



Gambar 2.3. Halaman Pundit AI

Pundit Controller

```
import 'package:get/get.dart';
import '../data/providers/api_provider.dart';

class PunditController extends GetxController {
  var isLoading = false.obs;
  var messages = <Map<String, String>>[].obs;

  Future<void> sendMessage(String text) async {
    final t = text.trim();
    if (t.isEmpty) return;

    messages.add({'role': 'user', 'text': t});
    isLoading.value = true;

    final prompt =
      'Anda adalah pundit sepak bola FootyHub. Jawab singkat, analitis,
    ramah. Pertanyaan: $t';
```

```

        final response = await ApiProvider.askGemini(prompt);

        if (response != null) {
            messages.add({'role': 'ai', 'text': response});
        } else {
            messages.add({
                'role': 'ai',
                'text': 'Maaf, layanan AI sedang sulit dijangkau. Cek kunci
Gemini.',
            });
        }

        isLoading.value = false;
    }
}

```

Pundit View

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';
import 'pundit_controller.dart';

class PunditView extends StatefulWidget {
    const PunditView({super.key});

    @override
    State<PunditView> createState() => _PunditViewState();
}

class _PunditViewState extends State<PunditView> {
    final _field = TextEditingController();
    final _scroll = ScrollController();

    @override

```

```

void dispose() {
  _field.dispose();
  _scroll.dispose();
  super.dispose();
}

void _send(PunditController c) {
  final text = _field.text;
  if (text.trim().isEmpty) return;
  c.sendMessage(text);
  _field.clear();
  Future.delayed(const Duration(milliseconds: 120), () {
    if (_scroll.hasClients) {
      _scroll.jumpTo(_scroll.position.maxScrollExtent);
    }
  });
}

@override
Widget build(BuildContext context) {
  final PunditController c = Get.find<PunditController>();

  return SafeArea(
    child: Column(
      children: [
        Padding(
          padding: const EdgeInsets.fromLTRB(20, 16, 20, 8),
          child: Row(
            children: [
              const CircleAvatar(
                backgroundColor: AppColors.primary,
                child: Icon(Icons.smart_toy_outlined, color: Colors.white),
              ),

```

```

const SizedBox(width: 12),
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      'Pundit AI',
      style: GoogleFonts.inter(
        fontSize: 18,
        fontWeight: FontWeight.bold,
        color: AppColors.primary,
      ),
    ),
    Text(
      'Gemini · tanya taktik, pemain, atau liga',
      style: GoogleFonts.inter(
        fontSize: 12,
        color: AppColors.tfPlaceholder,
      ),
    ),
  ],
),
Expanded(
  child: Obx(
    () => ListView.builder(
      controller: _scroll,
      padding:
        const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      itemCount: c.messages.length,
      itemBuilder: (_, i) {
        final m = c.messages[i];

```

```

final isUser = m['role'] == 'user';
return Align(
  alignment:
    isUser ? Alignment.centerRight : Alignment.centerLeft,
  child: Container(
    margin: const EdgeInsets.only(bottom: 8),
    padding: const EdgeInsets.symmetric(
      horizontal: 14, vertical: 10),
    constraints: BoxConstraints(
      maxWidth: MediaQuery.sizeOf(context).width * 0.82,
    ),
    decoration: BoxDecoration(
      color: isUser
        ? AppColors.seaGreen.withValues(alpha: 0.25)
        : AppColors.tfBackground,
      borderRadius: BorderRadius.only(
        topLeft: const Radius.circular(14),
        topRight: const Radius.circular(14),
        bottomLeft: Radius.circular(isUser ? 14 : 4),
        bottomRight: Radius.circular(isUser ? 4 : 14),
      ),
      border: Border.all(color: AppColors.tfBorder),
    ),
    child: Text(
      m['text'] ?? "",
      style: GoogleFonts.inter(
        fontSize: 14,
        height: 1.35,
        color: AppColors.textDark,
      ),
    ),
  ),
);

```

```

    },
  ),
),
),
Obx(
  () => c.isLoading.value
    ? const LinearProgressIndicator(minHeight: 2)
    : const SizedBox(height: 2),
),
Padding(
  padding: const EdgeInsets.fromLTRB(12, 8, 12, 12),
  child: Row(
    children: [
      Expanded(
        child: TextField(
          controller: _field,
          minLines: 1,
          maxLines: 4,
          decoration: InputDecoration(
            hintText: 'Tulis pertanyaan...',
            filled: true,
            fillColor:
              Theme.of(context).colorScheme.surfaceContainerHighest,
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(24),
              borderSide: BorderSide.none,
            ),
            contentPadding: const EdgeInsets.symmetric(
              horizontal: 18, vertical: 12),
          ),
          onSubmitted: (_) => _send(c),
        ),
      ),
    ],
  ),
),
),

```



```

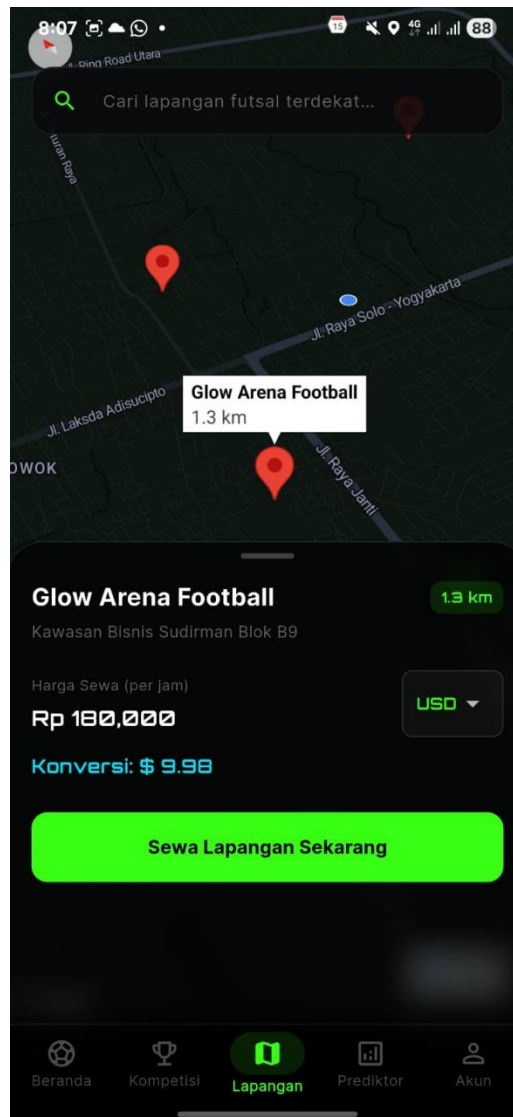
const SizedBox(width: 8),
IconButton.filled(
  onPressed: () => _send(c),
  icon: const Icon(Icons.send_rounded),
),
],
),
),
],
),
);
}
}

```

2.3.4. Fitur Empat : Map

Fitur maps memanfaatkan layanan peta untuk menampilkan lokasi tertentu serta rute perjalanan. Controller akan mengambil data lokasi pengguna melalui GPS, kemudian menentukan rute menuju lokasi tujuan.

View menampilkan peta interaktif yang dilengkapi dengan marker dan jalur perjalanan. Fitur ini sangat membantu pengguna dalam menemukan lokasi tertentu, seperti lapangan futsal terdekat.



Gambar 2.3. Halaman Map

Maps Controller

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:get/get.dart';
import 'package:geolocator/geolocator.dart';
import 'package:http/http.dart' as http;
import 'package:url_launcher/url_launcher.dart';
import '../core/constants/api_constants.dart';
import '../routes/app_routes.dart';

import 'package:flutter_polyline_points/flutter_polyline_points.dart';
```

```

class MapsController extends GetxController {
    var currentLatLng = const LatLng(-6.2000, 106.8166).obs;
    var markers = <Marker>{}.obs;
    var circles = <Circle>{}.obs;
    var polylines = <Polyline>{}.obs;
    var placeList = <dynamic>[].obs;
    var isLoading = false.obs;

    GoogleMapController? mapController;

    @override
    void onInit() {
        super.onInit();
        determinePosition();
    }

    String calculateDistance(double endLat, double endLng) {
        double distanceInMeters = Geolocator.distanceBetween(
            currentLatLng.value.latitude,
            currentLatLng.value.longitude,
            endLat,
            endLng,
        );
        return (distanceInMeters / 1000).toStringAsFixed(1);
    }

    void openDirections(double lat, double lng) async {
        isLoading.value = true;
        try {
            PolylinePoints polylinePoints = PolylinePoints(apiKey:
ApiConstants.googleMapsKey);
            PolylineResult result = await
polylinePoints.getRouteBetweenCoordinates(
                request: PolylineRequest(
                    origin: PointLatLng(currentLatLng.value.latitude,
currentLatLng.value.longitude),
                    destination: PointLatLng(lat, lng),
                    mode: TravelMode.driving,
                ),
            );
        }
    }
}

```

```

        if (result.points.isNotEmpty) {
            List<LatLng> polylineCoordinates = [];
            for (var point in result.points) {
                polylineCoordinates.add(LatLng(point.latitude,
point.longitude));
            }

            polylines.assign(
                Polyline(
                    polylineId: const PolylineId("route"),
                    color: Colors.blue,
                    points: polylineCoordinates,
                    width: 5,
                ),
            );

            // Adjust bounds
            double minLat = currentLatLng.value.latitude < lat ?
currentLatLng.value.latitude : lat;
            double maxLat = currentLatLng.value.latitude > lat ?
currentLatLng.value.latitude : lat;
            double minLng = currentLatLng.value.longitude < lng ?
currentLatLng.value.longitude : lng;
            double maxLng = currentLatLng.value.longitude > lng ?
currentLatLng.value.longitude : lng;

            LatLngBounds bounds = LatLngBounds(
                southwest: LatLng(minLat, minLng),
                northeast: LatLng(maxLat, maxLng),
            );

            mapController?.animateCamera(CameraUpdate.newLatLngBounds(bounds,
50));
        } else {
            Get.snackbar("Info", "Rute tidak ditemukan. Mungkin Anda berada
dalam mode offline.");
        }
    } catch (e) {
        Get.snackbar("Error", "Gagal menggambar rute: $e");
    } finally {
        isLoading.value = false;
    }
}

```

```

    }
}

void recenterOnUser() {
    determinePosition();
}

void determinePosition() async {
    isLoading.value = true;
    try {
        bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
        if (!serviceEnabled) {
            throw Exception("GPS mati. Mohon nyalakan GPS.");
        }

        LocationPermission permission = await Geolocator.checkPermission();
        if (permission == LocationPermission.denied) {
            permission = await Geolocator.requestPermission();
            if (permission == LocationPermission.denied) {
                throw Exception("Izin lokasi ditolak.");
            }
        }

        if (permission == LocationPermission.deniedForever) {
            await Geolocator.openAppSettings();
            throw Exception("Izin lokasi diblokir permanen.");
        }

        Position position = await Geolocator.getCurrentPosition(
            desiredAccuracy: LocationAccuracy.medium,
        ).timeout(
            const Duration(seconds: 12),
            onTimeout: () =>
                throw Exception("Gagal mendapat sinyal GPS."),
        );

        currentLatLng.value = LatLng(position.latitude,
position.longitude);
        mapController?.animateCamera(
            CameraUpdate.newLatLngZoom(currentLatLng.value, 14),
        );
    }
}

```

```

        circles.assign(
            Circle(
                circleId: const CircleId("radius_futsal"),
                center: currentLatLng.value,
                radius: 8000,
                fillColor: Colors.red.withOpacity(0.06),
                strokeColor: Colors.red.withOpacity(0.35),
                strokeWidth: 1,
            ),
        );

        fetchNearbyPlaces(position.latitude, position.longitude);
    } catch (e) {
        isLoading.value = false;
        Get.snackbar(
            "GPS",
            e.toString(),
            backgroundColor: Colors.red,
            colorText: Colors.white,
        );
    }
}

void searchPlaces(String query) {
    if (query.trim().isEmpty) {
        fetchNearbyPlaces(currentLatLng.value.latitude,
currentLatLng.value.longitude, keyword: "futsal");
    } else {
        fetchNearbyPlaces(currentLatLng.value.latitude,
currentLatLng.value.longitude, keyword: query.trim());
    }
}

void fetchNearbyPlaces(double lat, double lng, {String keyword =
"futsal"}) async {
    if (ApiConstants.googleMapsKey == 'MASUKKAN_GOOGLE_MAPS_KEY_ANDA') {
        await Future.delayed(const Duration(seconds: 1));
        Get.snackbar(
            "Mode Offline",

```

```

        "Menampilkan data dummy karena Google Maps API Key belum
dikonfigurasi.",
        backgroundColor: Colors.blue,
        colorText: Colors.white,
    );

    final dummyPlaces = [
        {
            'name': 'Futsal Arena (Dummy)',
            'vicinity': 'Jl. Dummy Futsal No. 1',
            'geometry': {
                'location': {'lat': lat + 0.01, 'lng': lng + 0.01}
            },
            'place_id': 'dummy_1'
        },
        {
            'name': 'Bintang Futsal (Dummy)',
            'vicinity': 'Jl. Bintang No. 99',
            'geometry': {
                'location': {'lat': lat - 0.01, 'lng': lng - 0.01}
            },
            'place_id': 'dummy_2'
        }
    ];

    final Set<Marker> newMarkers = {};
    for (var place in dummyPlaces) {
        final geometry = place['geometry'] as Map<String, dynamic>;
        final location = geometry['location'] as Map<String, dynamic>;
        final latToko = location['lat'] as double;
        final lngToko = location['lng'] as double;
        place['distance'] = calculateDistance(latToko, lngToko);
        final pid = place['place_id'] as String;

        newMarkers.add(
            Marker(
                markerId: MarkerId(pid),
                position: LatLng(latToko, lngToko),
                icon:
BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueRed),
                infoWindow: InfoWindow(

```

```

        title: place['name'] as String,
        snippet: "${place['distance']} km",
    ),
    onTap: () {
        Get.toNamed(
            Routes.FIELD_DETAIL,
            arguments: {
                'name': place['name'],
                'vicinity': place['vicinity'],
                'lat': latToko,
                'lng': lngToko,
                'priceIdr': 150000,
            },
        );
    },
),
);
}

placeList.value = dummyPlaces;
markers.assignAll(newMarkers);
isLoading.value = false;
return;
}

final String url =
    "https://maps.googleapis.com/maps/api/place/nearbysearch/json?location=$lat,$lng&radius=8000&keyword=$keyword&type=gym&key=${ApiConstants.googleMapsKey}";

try {
    var response = await http.get(Uri.parse(url));

    if (response.statusCode == 200) {
        var data = json.decode(response.body);

        if (data['status'] != "OK" && data['status'] != "ZERO_RESULTS") {
            Get.snackbar(
                "Maps API",
                data['error_message']?.toString() ??
data['status'].toString(),
                backgroundColor: Colors.orange,

```



```

        colorText: Colors.white,
        duration: const Duration(seconds: 4),
    );
}

var results = data['results'] as List? ?? [];
final Set<Marker> newMarkers = {};

for (var place in results) {
    final latToko = place['geometry']['location']['lat'];
    final lngToko = place['geometry']['location']['lng'];
    place['distance'] = calculateDistance(latToko, lngToko);
    final pid = place['place_id']?.toString() ??
'$latToko$lngToko';

    newMarkers.add(
        Marker(
            markerId: MarkerId(pid),
            position: LatLng(
                (latToko as num).toDouble(),
                (lngToko as num).toDouble(),
            ),
            icon: BitmapDescriptor.defaultMarkerWithHue(
                BitmapDescriptor.hueRed,
            ),
            infoWindow: InfoWindow(
                title: place['name']?.toString(),
                snippet: "${place['distance']} km",
            ),
            onTap: () {
                Get.toNamed(
                    Routes.FIELD_DETAIL,
                    arguments: {
                        'name': place['name']?.toString() ?? 'Lapangan',
                        'vicinity': place['vicinity']?.toString() ?? '',
                        'lat': latToko,
                        'lng': lngToko,
                        'priceIdr': 150000,
                    },
                );
            },
        ),
    );
}

```

```

        ),
    );
}

placeList.value = results;
markers.assignAll(newMarkers);
} else {
    Get.snackbar(
        "Server",
        "Gagal Places API: ${response.statusCode}",
    );
}
} catch (e) {
    Get.snackbar(
        "Jaringan",
        "Periksa koneksi / API key",
        backgroundColor: Colors.red,
        colorText: Colors.white,
    );
} finally {
    isLoading.value = false;
}
}
}

```

Maps View

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'maps_controller.dart';

class MapsView extends StatelessWidget {
    final bool showBack;

    const MapsView({super.key, this.showBack = true});

    @override
    Widget build(BuildContext context) {

```

```
final MapsController controller = Get.find<MapsController>();
```

```
return Scaffold(  
  body: Stack(  
    children: [  
      Obx(  
        () => GoogleMap(  
          initialCameraPosition: CameraPosition(  
            target: controller.currentLatLng.value,  
            zoom: 14,  
          ),  
          markers: controller.markers.toSet(),  
          circles: controller.circles.toSet(),  
          polylines: controller.polylines.toSet(),  
          myLocationEnabled: true,  
          myLocationButtonEnabled: false,  
          zoomControlsEnabled: false,  
          onMapCreated: (gController) {  
            controller.mapController = gController;  
            gController.animateCamera(  
              CameraUpdate.newLatLngZoom(  
                controller.currentLatLng.value,  
                14,  
              ),  
            );  
          },  
        ),  
      ),  
    ],  
  ),  
  if (showBack)  
    Positioned(  
      top: MediaQuery.paddingOf(context).top + 8,  
      left: 12,  
      child: Material(  

```

```

        color: Colors.white,
        shape: const CircleBorder(),
        child: IconButton(
          icon: const Icon(Icons.arrow_back, color: Colors.black87),
          onPressed: () => Get.back(),
        ),
      ),
    ),
    Positioned(
      top: MediaQuery.paddingOf(context).top + (showBack ? 60 : 16),
      left: 16,
      right: 16,
      child: Material(
        elevation: 4,
        borderRadius: BorderRadius.circular(16),
        color: Colors.white,
        child: TextField(
          decoration: InputDecoration(
            hintText: 'Cari lapangan futsal...',
            prefixIcon: const Icon(Icons.search),
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(16),
              borderSide: BorderSide.none,
            ),
            contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
          ),
          onSubmitted: (value) => controller.searchPlaces(value),
        ),
      ),
    ),
    Positioned(
      right: 16,
      bottom: showBack ? 280 : 280,

```

```

child: FloatingActionButton(
  heroTag: 'map_recenter',
  onPressed: controller.recenterOnUser,
  backgroundColor: Colors.white,
  foregroundColor: Colors.red.shade700,
  child: const Icon(Icons.my_location),
),
),
Align(
  alignment: Alignment.bottomCenter,
  child: Container(
    height: 260,
    padding: const EdgeInsets.fromLTRB(12, 12, 12, 12),
    decoration: const BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
      boxShadow: [
        BoxShadow(
          color: Colors.black12,
          blurRadius: 10,
          spreadRadius: 2,
        ),
      ],
    ),
  ),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Container(
        width: 40,
        height: 4,
        margin: const EdgeInsets.only(bottom: 12),
        decoration: BoxDecoration(
          color: Colors.grey[300],

```

```

        borderRadius: BorderRadius.circular(10),
      ),
    ),
    Text(
      'Lapangan futsal terdekat',
      style: Theme.of(context).textTheme.titleSmall,
    ),
    const SizedBox(height: 8),
    Expanded(
      child: Obx(
        () => controller.isLoading.value &&
          controller.placeList.isEmpty
        ? const Center(child: CircularProgressIndicator())
        : ListView.builder(
            itemCount: controller.placeList.length,
            itemBuilder: (context, index) {
              var toko = controller.placeList[index];
              var latToko =
                toko['geometry']['location']['lat'];
              var lngToko =
                toko['geometry']['location']['lng'];

              return Card(
                margin: const EdgeInsets.only(bottom: 8),
                child: ListTile(
                  leading: const CircleAvatar(
                    backgroundColor: Colors.red,
                    child: Icon(Icons.sports_soccer,
                      color: Colors.white),
                  ),
                  title: Text(
                    toko['name']?.toString() ?? "",
                    style: const TextStyle(

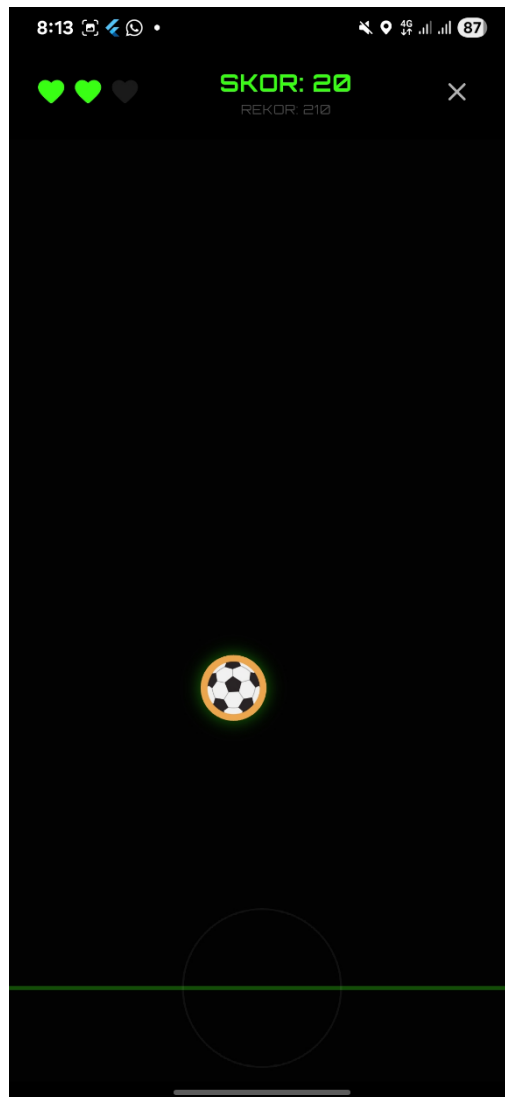
```

```
fontWeight: FontWeight.bold),
    ),
    subtitle: Text(
      toko['vicinity']?.toString() ?? ""),
    trailing: TextButton(
      onPressed: () =>
        controller.openDirections(
          (latToko as num).toDouble(),
          (lngToko as num).toDouble(),
        ),
      child: const Text('Rute'),
    ),
  ),
);
},
),
),
l,
),
),
),
l,
),
);
}
}
```

2.3.5. Fitur Lima : Game

Fitur game merupakan fitur hiburan dalam aplikasi. Controller mengatur logika permainan seperti skor, nyawa, serta aturan permainan.

View menampilkan tampilan game yang interaktif dan memungkinkan pengguna berinteraksi secara langsung. Fitur ini menambah daya tarik aplikasi karena tidak hanya bersifat informatif tetapi juga menghibur.



Gambar 2.3. Halaman Game

Game Controller

```
import 'dart:async';
import 'package:get/get.dart';
import 'package:light/light.dart';

// Import kedua pondasi yang sudah kita pastikan kokoh tadi
```



```

import '../core/utils/storage_util.dart'; // Sesuaikan path jika
berbeda
import '../data/locals/hive_provider.dart'; // Sesuaikan path jika
berbeda

class GameController extends GetxController {
  var score = 0.obs;
  var highScore = 0.obs;
  var hearts = 3.obs;
  var isGameOver = false.obs;

  var luxValue = 0.obs;
  late Light _light;
  StreamSubscription? _subscription;

  String? currentUsername;

  @override
  void onInit() {
    super.onInit();
    startListening();

    // Panggil fungsi untuk mengambil identitas dan skor saat game
    pertama kali dibuka
    _loadUserAndHighScore();
  }

  // Fungsi baru untuk mengambil data dari core dan data layer
  Future<void> _loadUserAndHighScore() async {
    // 1. Tanya ke StorageUtil: "Siapa yang sedang main sekarang?"
    currentUsername = await StorageUtil.getLoggedInUsername();

    // 2. Jika ada yang login, minta brankas skornya ke HiveProvider
    if (currentUsername != null) {
      highScore.value = HiveProvider.getHighScore(currentUsername!);
    }
  }

  void startListening() {
    _light = Light();
    try {

```

```

        _subscription = _light.lightSensorStream.listen((lux) {
            luxValue.value = lux;
        });
    } catch (e) {
        print("Sensor cahaya tidak ditemukan pada perangkat ini");
    }
}

double get nightOverlayOpacity {
    if (luxValue.value > 50) return 0.0;
    if (luxValue.value <= 5) return 0.6;
    return (50 - luxValue.value) / 100;
}

void increaseScore() {
    if (!isGameOver.value) {
        score.value += 10;

        // Jika pemain memecahkan rekornya sendiri...
        if (score.value > highScore.value) {
            highScore.value = score.value; // Update UI di layar seketika

            // 3. Simpan rekor baru ke HiveProvider agar permanen!
            if (currentUsername != null) {
                HiveProvider.saveHighScore(currentUsername!, score.value);
            }
        }
    }
}

void decreaseHeart() {
    if (!isGameOver.value && hearts.value > 0) {
        hearts.value--;
        if (hearts.value == 0) {
            isGameOver.value = true;
        }
    }
}

void resetGame() {
    score.value = 0;
}

```

```

        hearts.value = 3;
        isGameOver.value = false;
    }

    @override
    void onClose() {
        _subscription?.cancel();
        super.onClose();
    }
}

```

Game View

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';
import 'game_controller.dart';

class GameView extends StatefulWidget {
    const GameView({super.key});

    @override
    State<GameView> createState() => _GameViewState();
}

class GameViewState extends State<GameView> with
SingleTickerProviderStateMixin {
    late final AnimationController _anim;
    double _y = 0.12;
    double _vy = 0.001;
    double _x = 0.5;

    @override
    void initState() {
        super.initState();
        _anim = AnimationController(
            vsync: this,
            duration: const Duration(seconds: 1),

```

```

    )..repeat();
    _anim.addListener(_step);
}

void _step() {
    final gc = Get.find<GameController>();
    if (gc.isGameOver.value) return;
    setState(() {
        _vy += 0.0005; // Slower gravity
        _y += _vy;
        if (_y > 0.9) {
            _y = 0.1;
            _vy = 0.001;
            _x = 0.3 + (DateTime.now().millisecond % 100) / 250;
            gc.decreaseHeart();
        }
    });
}

void _tap(TapDownDetails d, Size size) {
    final gc = Get.find<GameController>();
    if (gc.isGameOver.value) return;
    final cx = _x * size.width;
    final cy = _y * size.height;
    if ((d.localPosition - Offset(cx, cy)).distance < 56) {
        setState(() {
            _vy = -0.012; // Slower jump
            _y = (_y - 0.06).clamp(0.08, 0.5);
            gc.increaseScore();
        });
    }
}

@override
void dispose() {
    _anim.dispose();
    super.dispose();
}

@override
Widget build(BuildContext context) {

```

```

final GameController gc = Get.find<GameController>();

return Scaffold(
  backgroundColor: AppColors.pureWhite,
  body: SafeArea(
    child: Obx(
      () => Stack(
        children: [
          Column(
            children: [
              Padding(
                padding: const EdgeInsets.symmetric(
                  horizontal: 16, vertical: 8),
                child: Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Row(
                      children: List.generate(
                        3,
                        (i) => Icon(
                          Icons.favorite,
                          color: i < gc.hearts.value
                            ? AppColors.dangerRed
                            : Colors.grey.shade300,
                        ),
                      ),
                    ),
                  ],
                ),
              Text(
                'Skor ${gc.score.value}',
                style: GoogleFonts.inter(
                  fontWeight: FontWeight.w800,
                  fontSize: 18,
                  color: AppColors.primary,
                ),
              ),
              IconButton(
                onPressed: () => Get.back(),
                icon: const Icon(Icons.close),
              ),
            ],
          ),
        ],
      ),
    ),
  ),
)

```

```

    ),
    Expanded(
      child: LayoutBuilder(
        builder: (context, cts) {
          return GestureDetector(
            behavior: HitTestBehavior.opaque,
            onTapDown: (d) => _tap(d, cts.biggest),
            child: Stack(
              children: [
                CustomPaint(
                  painter: _PitchPainter(),
                  size: Size.infinite,
                ),
                Positioned(
                  left: (_x * cts.maxWidth) - 25,
                  top: (_y * cts.maxHeight) - 25,
                  child: ClipOval(
                    child: Image.asset(
                      'assets/images/logobola.png',
                      width: 50,
                      height: 50,
                      fit: BoxFit.cover,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  ],
),
if (gc.isGameOver.value)
  Container(
    color: Colors.black54,
    alignment: Alignment.center,
    child: Card(
      margin: const EdgeInsets.all(32),
      child: Padding(
        padding: const EdgeInsets.all(24),

```

```

        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Text(
              'Game over',
              style: GoogleFonts.inter(
                fontSize: 22,
                fontWeight: FontWeight.bold,
                color: AppColors.dangerRed,
              ),
            ),
            const SizedBox(height: 12),
            Text(
              'Skor: ${gc.score.value}',
              style: GoogleFonts.inter(fontSize: 18),
            ),
            const SizedBox(height: 20),
            FilledButton(
              onPressed: () {
                gc.resetGame();
                setState(() {
                  _y = 0.12;
                  _vy = 0.001;
                  _x = 0.5;
                });
              },
              child: const Text('Main lagi'),
            ),
          ],
        ),
      ),
    ),
  ),
),
],
);
}
}

```

```

class _PitchPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size s) {
    final h = s.height;
    final w = s.width;
    final grass = Paint()..color = const Color(0xFFE8F5E9);
    canvas.drawRect(Offset.zero & s, grass);

    final line = Paint()
      ..color = Colors.white
      ..strokeWidth = 3;
    canvas.drawLine(Offset(0, h * 0.92), Offset(w, h * 0.92), line);
  }

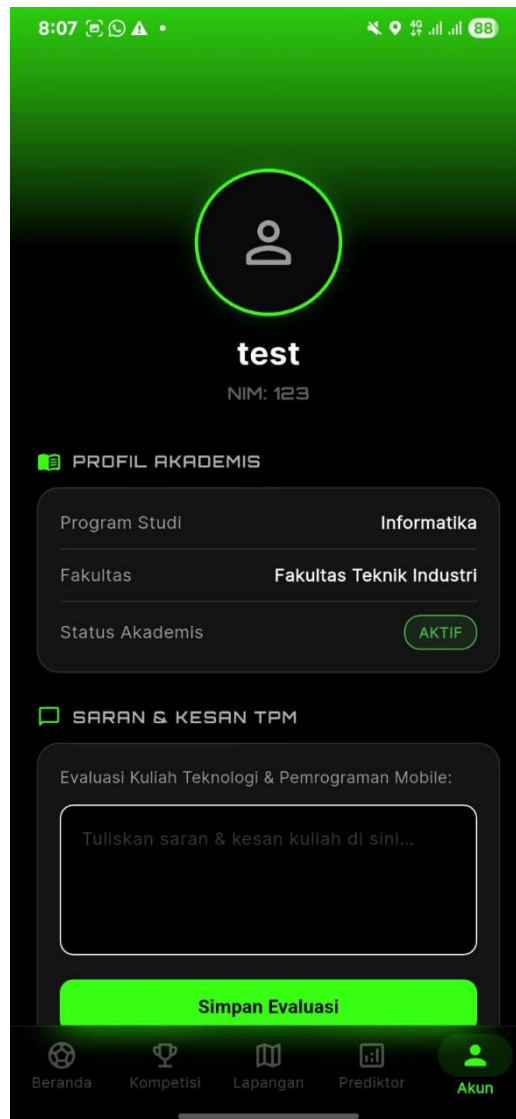
  @override
  bool shouldRepaint(covariant _PitchPainter oldDelegate) {
    return false;
  }
}

```

2.3.4. Fitur Enam : Profile

Fitur profile digunakan untuk menampilkan informasi pengguna yang telah login. Data pengguna diambil dari local storage yang sebelumnya disimpan saat proses login.

Controller mengelola data pengguna serta pengaturan akun, seperti aktivasi biometrik. View menampilkan informasi seperti nama, username, dan data lainnya dalam tampilan yang sederhana dan mudah dipahami.



Gambar 2.3. Halaman Profile

Profile Controller

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:local_auth/local_auth.dart';
import 'package:image_picker/image_picker.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../core/utils/storage_util.dart';
import '../data/locals/hive_provider.dart';
import '../data/locals/news_database.dart';
import '../routes/app_routes.dart';

class ProfileController extends GetxController {
  var currentName = 'Loading...'.obs;
}
```

```

var currentNim = '...'.obs;
String? accountUsername;

var currentProfileImagePath = ''.obs;
final ImagePicker _picker = ImagePicker();

final TextEditingController testimonialController =
TextEditingController();

var isBiometricEnabled = false.obs;
final LocalAuthentication auth = LocalAuthentication();

@override
void onInit() {
    super.onInit();
    _loadUserData();
}

Future<void> _loadUserData() async {
    accountUsername = await StorageUtil.getLoggedInUsername();

    final prefs = await SharedPreferences.getInstance();
    currentName.value =
        prefs.getString(StorageUtil.keyUserName) ?? 'User Unknown';
    currentNim.value = prefs.getString(StorageUtil.keyUserNim) ??
'000000';

    if (accountUsername != null) {
        isBiometricEnabled.value =
            HiveProvider.getBiometricStatus(accountUsername!);
        currentProfileImagePath.value =
            HiveProvider.getProfileImagePath(accountUsername!);
        testimonialController.text =
            HiveProvider.getTestimonialContent(accountUsername!);
    }
}

Future<void> pickProfileImage() async {
    if (accountUsername == null) {
        Get.snackbar('Error', 'Sesi tidak valid.');
```

```

    }

    try {
        final XFile? image = await _picker.pickImage(
            source: ImageSource.gallery,
            imageQuality: 50,
            maxWidth: 800,
        );

        if (image != null) {
            currentProfileImagePath.value = image.path;
            HiveProvider.saveProfileImagePath(accountUsername!, image.path);
        }
    } catch (e) {
        Get.snackbar('Error', 'Gagal memuat gambar galeri.');
```

```

    }

    Future<void> submitTpmFeedback() async {
        if (accountUsername == null) return;
        HiveProvider.saveTestimonial(
            accountUsername!,
            testimonialController.text,
            5,
        );
        Get.snackbar(
            'Terkirim',
            'Saran & kesan TPM telah disimpan.',
            backgroundColor: Colors.green.shade700,
            colorText: Colors.white,
        );
    }

```

```

    Future<void> toggleBiometric(bool value) async {
        if (accountUsername == null) return;

        if (value == true) {
            try {
                bool canCheck = await auth.canCheckBiometrics;
                bool isSupported = await auth.isDeviceSupported();

                if (canCheck && isSupported) {

```

```

        bool didAuthenticate = await auth.authenticate(
          localizedReason:
            'Aktifkan biometrik untuk login cepat FootyHub',
          biometricOnly: true,
        );

        if (didAuthenticate) {
          isBiometricEnabled.value = true;
          HiveProvider.saveBiometricStatus(accountUsername!, true);
        } else {
          isBiometricEnabled.value = false;
        }
      } else {
        Get.snackbar('Info', 'Perangkat tidak mendukung biometrik');
        isBiometricEnabled.value = false;
      }
    } catch (e) {
      isBiometricEnabled.value = false;
    }
  } else {
    isBiometricEnabled.value = false;
    HiveProvider.saveBiometricStatus(accountUsername!, false);
  }
}

Future<void> logout() async {
  await NewsDatabase.instance.clearAll();
  await StorageUtil.clearSession();
  Get.offAllNamed(Routes.AUTH);
}
}

```

Profile View

```

import 'dart:io';
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:google_fonts/google_fonts.dart';
import '../core/theme/app_colors.dart';

```

```

import 'profile_controller.dart';

class ProfileView extends StatelessWidget {
  const ProfileView({super.key});

  @override
  Widget build(BuildContext context) {
    final ProfileController controller = Get.put(ProfileController());

    return Scaffold(
      backgroundColor: AppColors.pureWhite,
      body: SingleChildScrollView(
        child: Column(
          children: [
            // - Header -
            Stack(
              clipBehavior: Clip.none,
              alignment: Alignment.center,
              children: [
                Container(
                  width: double.infinity,
                  height: 180,
                  decoration: const BoxDecoration(color: AppColors.primary),
                ),
                // Avatar Lingkaran
                Positioned(
                  bottom: -50,
                  child: GestureDetector(
                    onTap: controller
                      .pickProfileImage,
                    child: Obx(
                      () => Container(
                        width: 110,
                        height: 110,
                        decoration: BoxDecoration(
                          color: AppColors.primary,
                          shape: BoxShape.circle,
                          border: Border.all(
                            color: AppColors.pureWhite,
                            width: 4,
                          ),
                        ),
                      ),
                    ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.1),
            blurRadius: 8,
            offset: const Offset(0, 4),
          ),
        ],
        image:
          controller
            .currentProfileImagePath
            .value
            .isNotEmpty
          ? DecorationImage(
              image: FileImage(
                File(
                  controller.currentProfileImagePath.
value,
                ),
              ),
              fit: BoxFit.cover,
            )
          : null,
      ),
    ),
  ),
  Positioned(
    bottom: -45,
    right: MediaQuery.of(context).size.width / 2 - 55,
    child: Container(
      padding: const EdgeInsets.all(6),
      decoration: const BoxDecoration(
        child:
          controller.currentProfileImagePath.value.isEmpty
            ? const Icon(
                Icons.person_outline_rounded,
                size: 50,
                color: AppColors.pureWhite,
              )
            : null,
      ),
    ),
  ),
),
Positioned(
  bottom: -45,
  right: MediaQuery.of(context).size.width / 2 - 55,
  child: Container(
    padding: const EdgeInsets.all(6),
    decoration: const BoxDecoration(

```

```

        color: AppColors.seaGreen,
        shape: BoxShape.circle,
      ),
      child: const Icon(
        Icons.camera_alt_rounded,
        color: Colors.white,
        size: 16,
      ),
    ),
  ],
),
const SizedBox(height: 60),

// --- NAMA DAN NIM DINAMIS MENGGUNAKAN Obx ---
Obx(
  () => Text(
    controller.currentName.value,
    style: GoogleFonts.inter(
      fontSize: 24,
      fontWeight: FontWeight.bold,
      color: AppColors.primary,
    ),
  ),
),
const SizedBox(height: 6),
Obx(
  () => Text(
    'Student ID: ${controller.currentNim.value}',
    style: GoogleFonts.inter(
      fontSize: 14,
      color: AppColors.tfPlaceholder,
    ),
  ),
),
const SizedBox(height: 32),

Padding(
  padding: const EdgeInsets.symmetric(horizontal: 24),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,

```

```

        children: [
          _buildSectionHeader(
            Icons.menu_book_rounded,
            'Academic Profile',
            AppColors.oceanTeal.withValues(alpha: 0.1),
            AppColors.oceanTeal,
          ),
          const SizedBox(height: 12),
          Container(
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: AppColors.pureWhite,
              borderRadius: BorderRadius.circular(16),
              border: Border.all(color: AppColors.tfBorder),
            ),
            child: Column(
              children: [
                _buildProfileRow('Program', 'Informatika'),
                const Padding(
                  padding: EdgeInsets.symmetric(vertical: 12),
                  child: Divider(height: 1, color:
AppColors.tfBorder),
                ),
                _buildProfileRow('Faculty', 'Teknik Industri'),
                const Padding(
                  padding: EdgeInsets.symmetric(vertical: 12),
                  child: Divider(height: 1, color:
AppColors.tfBorder),
                ),
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Text(
                      'Status',
                      style: GoogleFonts.inter(
                        fontSize: 14,
                        color: AppColors.tfPlaceholder,
                      ),
                    ),
                    Container(
                      padding: const EdgeInsets.symmetric(

```



```

        horizontal: 12,
        vertical: 6,
      ),
      decoration: BoxDecoration(
        color: Colors.green.withValues(alpha: 0.1),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: Colors.green.withValues(alpha:
0.3),

        ),
      ),
      child: Text(
        'Active',
        style: GoogleFonts.inter(
          fontSize: 12,
          fontWeight: FontWeight.w600,
          color: Colors.green,
        ),
      ),
    ],
  ),
],
),
),
const SizedBox(height: 24),

_buildSectionHeader(
  Icons.chat_bubble_outline_rounded,
  'Saran & kesan TPM',
  AppColors.dangerRed.withValues(alpha: 0.1),
  AppColors.dangerRed,
),
const SizedBox(height: 12),
Container(
  padding: const EdgeInsets.all(20),
  decoration: BoxDecoration(
    color: AppColors.pureWhite,
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: AppColors.tfBorder),
  ),

```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      'Evaluasi untuk dosen pengampu',
      style: GoogleFonts.inter(
        fontSize: 13,
        color: AppColors.tfPlaceholder,
      ),
    ),
    const SizedBox(height: 12),
    Container(
      padding: const EdgeInsets.symmetric(horizontal:
16),

      decoration: BoxDecoration(
        color: AppColors.tfBackground,
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: AppColors.tfBorder),
      ),
      child: TextField(
        controller: controller.testimonialController,
        maxLines: 5,
        minLines: 3,
        style: GoogleFonts.inter(
          fontSize: 14,
          color: AppColors.textDark,
        ),
        decoration: InputDecoration(
          hintText:
            'Tuliskan saran dan kesan Anda di sini...',
          hintStyle: GoogleFonts.inter(
            color: AppColors.tfPlaceholder,
          ),
          border: InputBorder.none,
        ),
      ),
    ),
    const SizedBox(height: 16),
    SizedBox(
      width: double.infinity,
      child: FilledButton(

```

```

        onPressed: controller.submitTpmFeedback,
        style: FilledButton.styleFrom(
          backgroundColor: AppColors.primary,
          padding: const EdgeInsets.symmetric(vertical:
16),
        ),
        child: Text(
          'Kirim Evaluasi',
          style: GoogleFonts.inter(
            fontWeight: FontWeight.w600,
            color: Colors.white,
          ),
        ),
      ),
    ],
  ),
),
const SizedBox(height: 24),

_buildSectionHeader(
  Icons.settings_outlined,
  'App Settings',
  AppColors.seaGreen.withValues(alpha: 0.1),
  AppColors.seaGreen,
),
const SizedBox(height: 12),
Container(
  padding: const EdgeInsets.symmetric(
    horizontal: 20,
    vertical: 12,
  ),
  decoration: BoxDecoration(
    color: AppColors.pureWhite,
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: AppColors.tfBorder),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Row(

```

```

        children: [
          const Icon(
            Icons.fingerprint_rounded,
            color: AppColors.tfPlaceholder,
            size: 20,
          ),
          const SizedBox(width: 12),
          Text(
            'Login Biometric',
            style: GoogleFonts.inter(
              fontSize: 14,
              color: AppColors.primary,
            ),
          ),
        ],
      ),
      Obx(
        () => Switch(
          value: controller.isBiometricEnabled.value,
          onChanged: controller.toggleBiometric,
          activeThumbColor: AppColors.pureWhite,
          activeTrackColor: AppColors.seaGreen,
          inactiveThumbColor: AppColors.tfPlaceholder,
          inactiveTrackColor: AppColors.tfBorder,
        ),
      ),
    ],
  ),
  const SizedBox(height: 32),

  SizedBox(
    width: double.infinity,
    child: FilledButton(
      onPressed: controller.logout,
      style: FilledButton.styleFrom(
        backgroundColor: AppColors.dangerRed,
        padding: const EdgeInsets.symmetric(vertical: 22),
      ),
    ),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.center,

```

```

        children: [
            const Icon(Icons.logout_rounded, color:
Colors.white),

            const SizedBox(width: 10),
            Text(
                'Logout',
                style: GoogleFonts.inter(
                    fontWeight: FontWeight.bold,
                    color: Colors.white,
                    letterSpacing: 0.5,
                ),
            ),
        ],
    ),
    const SizedBox(height: 100),
],
),
),
),
const SizedBox(height: 100),
],
),
),
),
),
),
);
}

Widget _buildSectionHeader(
    IconData icon,
    String title,
    Color bgColor,
    Color iconColor,
) {
    return Row(
        children: [
            Container(
                padding: const EdgeInsets.all(8),
                decoration: BoxDecoration(
                    color: bgColor,
                    borderRadius: BorderRadius.circular(8),
                ),
                child: Icon(icon, color: iconColor, size: 18),
            ),
        ],
    );
}

```

```

    ),
    const SizedBox(width: 12),
    Text(
      title,
      style: GoogleFonts.inter(
        fontSize: 16,
        fontWeight: FontWeight.bold,
        color: AppColors.primary,
      ),
    ),
  ],
);
}

Widget _buildProfileRow(String label, String value) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(
        label,
        style: GoogleFonts.inter(
          fontSize: 14,
          color: AppColors.tfPlaceholder,
        ),
      ),
      Text(
        value,
        style: GoogleFonts.inter(
          fontSize: 14,
          fontWeight: FontWeight.w600,
          color: AppColors.primary,
        ),
      ),
    ],
  );
}
}

```

BAB III

TESTING

3.1. Metode Testing

Pengujian berikut dilakukan dengan metode Black Box Testing, yaitu metode pengujian yang berfokus pada fungsionalitas sistem tanpa memperhatikan struktur kode program di dalamnya. Pengujian ini bertujuan untuk memastikan bahwa setiap fitur dalam aplikasi *FootyHub* berjalan sesuai dengan kebutuhan dan menghasilkan output yang benar berdasarkan input yang diberikan oleh pengguna.

Dalam pengujian ini, setiap fitur diuji dengan berbagai skenario, seperti input valid, input tidak valid, serta kondisi khusus lainnya untuk melihat bagaimana sistem merespon. Hasil dari pengujian akan menunjukkan apakah fitur tersebut berjalan dengan baik atau masih terdapat kesalahan yang perlu diperbaiki.

Pengujian dilakukan pada beberapa fitur utama, antara lain fitur autentikasi (login, register, dan biometrik), dashboard, berita (news), encyclopedia, AI assistant, konversi, maps, game, serta profil pengguna. Dengan pengujian ini, diharapkan aplikasi dapat berjalan secara optimal, stabil, dan sesuai dengan kebutuhan pengguna.

3.2. Rencana dan Prosedur Testing

Pengujian dilakukan terhadap beberapa fitur utama berikut:

No	Fitur Diuji	Skenario Pengujian	Hasil yang Diharapkan	Evaluasi
1	Autentikasi (Email/Password)	Input kredensial yang valid dan klik masuk.	Aplikasi sukses melakukan verifikasi dan memunculkan Dashboard.	[] Sukses / [] Gagal
2	Autentikasi Biometrik	Menekan ikon <i>Fingerprint</i> dan menempelkan sidik jari ke sensor.	Layar langsung terbuka dan <i>login bypass</i> berhasil dilakukan.	[] Sukses / [] Gagal
3	Navigasi Utama	Membuka <i>Home</i> dan <i>Encyclopedia</i> bergantian.	Konten termuat penuh melalui API tanpa <i>lag/crash</i> .	[] Sukses / [] Gagal

4	Modul Tools & Konversi	Memasukkan nominal angka di kalkulator uang.	Konversi mata uang tepat dan up-to-date berdasarkan rate API.	☐ Sukses / ☐ Gagal
5	Modul Maps	Menetik "Gelora Bung Karno" pada <i>Search Box</i> .	Peta bergeser ke koordinat dan memunculkan pin <i>marker</i> stadion.	☐ Sukses / ☐ Gagal
6	Chatbot AI Pundit	Mengirim prompt "Siapa pemenang piala dunia 2022?"	Chatbot <i>Cohere</i> merespons secara informatif dan relevan.	☐ Sukses / ☐ Gagal
7	Sensor Hardware (<i>Accelerometer</i>)	Melakukan gerakan mengocok/shake pada smartphone.	Antarmuka tertrigger untuk merefresh halaman / menjalankan aksi tertentu.	☐ Sukses / ☐ Gagal
8	Sensor Hardware (<i>Proximity</i>)	Menutup ujung sensor di dekat kamera depan.	Layar masuk mode proteksi sentuhan / redup secara otomatis.	☐ Sukses / ☐ Gagal
9	Manajemen Sesi	Merestart aplikasi (<i>force close</i> dan buka kembali).	Aplikasi masuk otomatis tanpa melalui halaman login kembali.	☐ Sukses / ☐ Gagal
10	Logout Aplikasi	Menekan opsi "Keluar" dari tab profil.	Akses diputus, memori lokal dibersihkan, dan kembali ke <i>Screen Login</i> .	☐ Sukses / ☐ Gagal

Tabel 3.1. Rencana dan Prosedur Testing

BAB IV

KESIMPULAN

4.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi FootyHub berhasil dikembangkan sebagai aplikasi mobile berbasis Flutter yang mampu mengintegrasikan berbagai fitur dalam satu platform. Aplikasi ini menyediakan layanan informasi, utilitas, serta hiburan, yang meliputi fitur berita, peta lokasi, ensiklopedia, konversi, permainan, serta asisten berbasis kecerdasan buatan.

Penerapan arsitektur Model-View-Controller (MVC) yang dikombinasikan dengan framework GetX memberikan struktur yang jelas antara komponen data, logika, dan tampilan, sehingga meningkatkan keteraturan kode serta memudahkan proses pengembangan dan pemeliharaan sistem. Selain itu, implementasi sistem autentikasi berbasis JSON Web Token (JWT) yang didukung dengan fitur biometrik turut meningkatkan aspek keamanan dan kenyamanan pengguna.

Berdasarkan hasil pengujian menggunakan metode Black Box Testing, seluruh fitur utama dalam aplikasi telah berjalan sesuai dengan fungsionalitas yang dirancang. Dengan demikian, aplikasi ini dapat dikatakan telah memenuhi kebutuhan sistem yang telah ditetapkan dan layak untuk digunakan sebagai aplikasi mobile yang terintegrasi.

4.2. Saran

Adapun beberapa saran yang dapat diberikan untuk pengembangan aplikasi di masa mendatang adalah sebagai berikut:

1. Perlu dilakukan pengembangan lebih lanjut pada aspek antarmuka pengguna (UI/UX) agar tampilan aplikasi menjadi lebih menarik, modern, dan responsif.
2. Penambahan fitur notifikasi secara real-time (push notification) guna meningkatkan interaksi dan keterlibatan pengguna terhadap aplikasi.
3. Pengembangan sistem penyimpanan berbasis cloud untuk meningkatkan fleksibilitas dan keamanan data.
4. Peningkatan sistem keamanan, seperti pengelolaan sesi yang lebih optimal serta penambahan lapisan enkripsi pada data pengguna.
5. Integrasi dengan layanan autentikasi pihak ketiga, seperti akun Google, untuk mempermudah proses login pengguna.

6. Optimalisasi performa aplikasi agar dapat berjalan lebih ringan dan efisien pada berbagai perangkat.
7. Pengembangan fitur tambahan yang disesuaikan dengan kebutuhan pengguna di masa yang akan datang.

LAMPIRAN

LAMPIRAN : Source Code Github

Github : <https://github.com/xProbe/footyhub/tree/main>

Lampiran ini adalah sumber kode yang telah dibuat pada proyek ini. Untuk hasil pengerjaan bisa diakses pada link yang tersedia pada Lampiran ini.